

Linux Infrastructure Intern Take-Home Assignment

Project: Server Provisioning, Automation, and Basic Hardening Mini Lab

1. Assignment Objective

Build a small, reproducible Linux provisioning project that converts a fresh local virtual machine into a deployment-ready server environment. The assignment tests practical Linux administration, shell scripting, systemd usage, package/service configuration, basic hardening, reboot validation, and documentation quality. The entire assignment must be completed inside a local VM only, not on any cloud VM.

2. Candidate Environment

- Use Ubuntu Server 22.04/24.04 LTS, Debian 12, Rocky Linux, or another mainstream Linux distribution.
- Run the assignment inside a local VM only using VirtualBox, VMware Workstation/Player, Hyper-V, UTM, or a similar local virtualization tool.
- Do not use AWS, Azure, Google Cloud, DigitalOcean, Linode, Oracle Cloud, or any other cloud VM/server for this assignment.
- All screenshots, logs, validation outputs, and demo video should clearly come from the local VM environment.
- Do not run destructive commands on your personal host machine. Keep all scripts scoped to the local VM environment.
- Use Bash as the primary automation language. Python may be used for validation or helper scripts.
- Git must be used from the beginning. Commit your work in meaningful milestones.

3. Use of AI / LLM Tools

- Use of AI tools is allowed for research, boilerplate, command explanation, and documentation help.
- You must understand every command and configuration you submit.
- Include a short section in the README named “AI Assistance Notes” describing where AI helped and what you manually verified.

4. Scenario

You are asked to prepare a Linux server baseline inside a local VM for internal deployment practice. The server should install required packages, configure a small demo service, apply basic security hardening, set up maintenance automation, and provide a validation script that proves the system works after provisioning and reboot. No cloud VM, cloud image, or cloud provider account is required or allowed for this assignment.

5. Required Project Components

Component	Purpose	Expected Path
Provisioning script	Installs packages, creates users/directories, configures service files, applies safe defaults.	scripts/provision.sh
Validation script	Checks service health, firewall rules, users, permissions, logs, and reboot survival.	scripts/validate.sh

Demo service	A lightweight service managed by systemd. Can be nginx or a simple Python HTTP health service.	systemd/infra-demo.service
Environment file	Stores non-secret runtime config such as port and log path.	config/infra-demo.env
Maintenance timer	Runs a simple periodic task such as log cleanup or health snapshot collection.	systemd/infra-maintenance.timer
Hardening checklist	Documents security settings applied and intentionally not applied.	docs/hardening-checklist.md
Test evidence	Screenshots/logs showing each milestone and final reboot validation.	evidence/
README	Setup steps, architecture, commands, assumptions, troubleshooting, and demo instructions.	README.md

6. Functional Requirements

- **FR1 - Base setup:** Detect OS/distribution, update package index safely, install required packages, set hostname/timezone if documented, and create a non-root sudo user for operational work.
- **FR2 - Service setup:** Install and configure either nginx or a small Python-based HTTP health service. The service must be managed by systemd and start automatically on boot.
- **FR3 - Logs and config:** Use an environment file for service configuration. Ensure logs are visible through journalctl and, if applicable, written to a controlled log directory.
- **FR4 - Automation quality:** Provisioning should be repeatable. Running the script twice should not break the system or duplicate users/configuration unexpectedly.
- **FR5 - Basic hardening:** Apply checklist-based hardening: SSH safe defaults, firewall rules, package updates, restricted permissions, disabling unused services where safe, and secure file ownership.
- **FR6 - Local reprovisioning flow: Provide clear local VM reprovisioning instructions. This may include creating a fresh VM, taking a local snapshot before provisioning, restoring the snapshot, and running the provisioning script again to simulate image customization and redeployment. Use only local VM snapshot/clone/reset-and-rerun workflow.**
- **FR7 - Validation:** The validation script must check service status, HTTP health response, open ports, firewall state, file permissions, user existence, and recent logs.
- **FR8 - Reboot survival:** Demonstrate that after a reboot the service comes back online and validation still passes.

7. Suggested Repository Structure

```

linux-infra-intern-assignment/
├── README.md
├── scripts/
│   ├── provision.sh
│   ├── validate.sh
│   └── maintenance.sh
├── systemd/
│   ├── infra-demo.service
│   ├── infra-maintenance.service
│   └── infra-maintenance.timer
├── config/
│   └── infra-demo.env
├── docs/
│   ├── hardening-checklist.md
│   └── local-vm-reprovisioning.md

```

```

├── test-plan.md
├── troubleshooting.md
├── evidence/
│   ├── milestone-1-setup.png
│   ├── milestone-2-service.png
│   ├── milestone-3-hardening.png
│   └── final-reboot-validation.png

```

8. Milestones

Milestone	Expected Output	Evidence
Base VM + repo setup	Fresh local VM Linux environment, Git repo, initial README, package plan, basic provision.sh skeleton.	Screenshot of OS/version, repo structure, first successful script run.
Service + systemd	Demo service installed, systemd unit enabled, health endpoint or nginx page working.	systemctl status, curl output, journalctl output.
Hardening + automation	Firewall configured, SSH defaults reviewed, secure permissions, maintenance timer added, script made idempotent.	Firewall status, timer status, permissions check, second script run output.
Validation + reboot testing	validate.sh covers core checks; reboot survival tested; troubleshooting notes added.	Validation output before and after reboot.
Cleanup + documentation + demo	README polished, test evidence organized, demo video recorded, stretch goal if time permits.	Final repo, demo link, screenshots/logs.

9. Minimum Validation Checks

Check Area	Minimum Check	Example Command / Evidence
Service	Service is enabled and active.	systemctl is-enabled infra-demo; systemctl is-active infra-demo
Health	HTTP endpoint returns expected response/status.	curl -i http://localhost:8080/health or curl -i http://localhost
Firewall	Only expected ports are open.	ufw status or firewall-cmd --list-all; ss -tulpn
Users/permissions	Operational user exists; config files have safe ownership and mode.	id <user>; stat -c "%U %G %a %n" <file>
Logs	Recent service logs are readable and meaningful.	journalctl -u infra-demo --no-pager -n 30
Reboot	Service survives reboot.	uptime + validate.sh after reboot
Idempotency	Provisioning can run twice safely.	Run provision.sh twice and show no

destructive side effects.

10. Stretch Goals

- Add an Ansible playbook equivalent to your Bash provisioning flow.
- Add local VM snapshot and restore instructions that show how to repeat the setup from a clean VM state.
- Add optional local-only VM image notes, such as how to export/import an appliance from VirtualBox or document a reusable local VM template.
- Add a simple monitoring endpoint or node_exporter setup notes.
- Add Docker deployment of the demo service, while still keeping systemd and host hardening clear.
- Add rollback or uninstall script with safety checks.

11. Deliverables

1. Public or shared GitHub repository containing the final project.
2. README with setup steps, assumptions, supported OS, command examples, validation steps, troubleshooting, and AI Assistance Notes.
3. 1-3 minute demo video link showing provisioning, service health, validation, and reboot survival evidence.
4. Screenshots or terminal logs for each milestone under the evidence/ folder.
5. Hardening checklist explaining what was applied, why it was applied, and what was intentionally skipped for safety.
6. Clear commit history with meaningful messages.

12. Evaluation Rubric

Area	Weight	What We Look For
Linux fundamentals	15%	Correct use of users, permissions, packages, services, logs, networking, and common CLI tools.
Automation quality	15%	Readable, idempotent Bash scripts with checks, clear output, and safe failure handling.
systemd usage	15%	Correct service/timer files, environment file usage, enable/start behavior, journal logs.
Security hardening	15%	Practical SSH/firewall/permissions/service restrictions with clear reasoning.
Validation and reliability	15%	validate.sh proves the setup works before/after reboot and reports failures clearly.
Documentation	15%	README and docs are clear enough for another engineer to reproduce the setup.
Demo and evidence	10%	Concise demo and organized proof for each milestone.

13. Important Safety Restrictions

- Do not create/run scripts that may delete disks, reformat partitions, or modify the host outside the local test VM.
- Do not commit private keys, passwords, credentials, tokens, or personal secrets.
- Do not disable SSH access unless you have console access and clearly document rollback steps.
- Do not submit screenshots containing secrets or private IPs that should not be public.

14. Final Submission Checklist

- Repository link is accessible.
- README contains one-command or clearly sequenced setup instructions.
- provision.sh and validate.sh run successfully on a fresh test environment.
- Systemd service and timer files are included.
- Hardening checklist is included.
- Evidence folder contains milestone screenshots/logs.
- Demo video link is present in README.
- No secrets are committed.
- Submission does not use or reference any cloud VM/provider; all work is done in a local VM.